



Bilgisayar Programlama

İleri Seviye Fonksiyonlar (2. Kısım)

İçindekiler

1. Pointer'lar ve Referans ile Çağırma (Pass by Reference)
 - Bellek adresi aktarımı, birden fazla değer döndürme, RAM tasarrufu
2. Bellek Sınıfları ve Modülerlik: `static` ve `extern`
 - Durum koruma, izole kod yazımı, çoklu dosya veri paylaşımı
3. Özylenelemeli (Recursive) Fonksiyonlar
 - Kendi kendini çağırma, Stack Overflow riski, iteratif dönüşüm
4. Yüksek Performanslı Çağrılar: `inline` Fonksiyonlar
 - Call Stack overhead, hız-boyut dengesi

İçindekiler (Devam)

5. Ön İşlemci Seviyesinde Optimizasyon: Makro Fonksiyonlar (`#define`)

- Parametrelili makrolar, tür güvenliği riskleri, register bit işlemleri

6. Esnek Veri Yapıları: Değişken Sayıda Parametre (Variadic Functions)

- `stdarg.h` kütüphanesi, dinamik hata loglaması

7. Dinamik Mimari: Fonksiyon İşaretçileri (Function Pointers)

- Callback fonksiyonlar, State Machine tasarımı

8. Dış Dünya ile İletişim: `main` Fonksiyonuna Parametre Geçme

- `argc` , `argv` yapısı, donanım port bilgisi aktarımı

BÖLÜM 1

Pointer'lar ve Referans ile Çağırma (Pass by Reference)

1.1 Call by Value'nun Sınırı (Hatırlatma)

C dilinde fonksiyonlara argüman gönderildiğinde **kopya** aktarılır. Fonksiyon içindeki değişiklik orijinali etkilemez.

× Sorun: Bir motor kontrol fonksiyonunun aynı anda konum, hız ve sıcaklık gibi birden fazla değeri güncellemesi gerekirse ne yapacağız? `return` ile yalnızca tek bir değer döndürülebilir.

```
1 void ikiyeKatla(int sayi) {
2     sayi = sayi * 2; // Yalnızca KOPYA değişir
3 }
4
5 int main() {
6     int x = 10;
7     ikiyeKatla(x);
8     printf("x = %d\n", x); // x hâlâ 10 – orijinal değişmedi!
9     return 0;
10 }
```

Çözüm: Fonksiyona değerini kendisini değil, bellek adresini (Pointer) göndermek.

1.2 Pointer (İşaretçi) Nedir?

Pointer, bir değişkenin RAM'deki adresini tutan özel bir değişkendir.

1 RAM Bellek Görünümü:

2
3
4
5
6
7

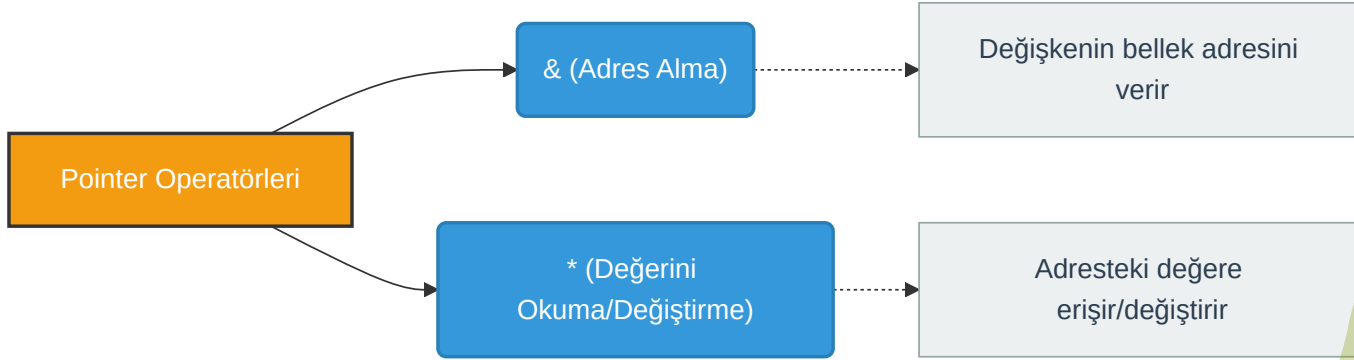
Adres	Değişken	Değer
0x1000	x	10
0x1008	ptr	0x1000

← x'in adresini tutar

```
1 int x = 10;
2 int *ptr = &x; // ptr, x'in adresini tutar
3
4 printf("x'in degeri : %d\n", x); // 10
5 printf("x'in adresi : %p\n", &x); // 0x1000 (örnek)
6 printf("ptr'nin degeri: %p\n", ptr); // 0x1000 (aynı adres)
7 printf("ptr ile erisim: %d\n", *ptr); // 10 (dereference)
```

1.2.1 Pointer (İşaretçi) Operatörleri

Operatör	Adı	İşlevi
&	Adres Alma	Değişkenin bellek adresini verir
*	Değerini Okuma/Değiştirme	Adresteki değere erişir/değiştirir



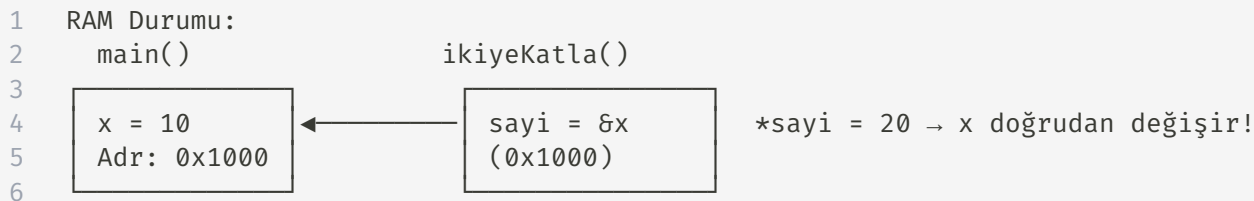
- Bu operatörler adres ile işlem yapmamıza olanak tanır.
- Özellikle donanım kontrolü, bellek yönetimi ve veri transferi gibi alanlarda sıklıkla kullanılır.

1.3 Pass by Reference — Adres ile Çağırma

Fonksiyona değişkenin adresini göndererek orijinal değeri doğrudan değiştirebiliriz.

✓ Çözüm: Pointer parametresi ile çağırma:

```
1 void ikiyeKatla(int *sayi) { // Adres alır (pointer parametresi)
2     *sayi = (*sayi) * 2;     // Adresteki değeri doğrudan değiştirir
3 }
4
5 int main() {
6     int x = 10;
7     ikiyeKatla(&x);          // x'in ADRESİNİ gönder
8     printf("x = %d\n", x);    // x artık 20!
9     return 0;
10 }
```



1.4 Birden Fazla Değer Döndürme Problemi

Bir motor sensöründen aynı anda hız, sıcaklık ve akım okumak istiyoruz. `return` ile yalnızca tek değer döndürülebilir. Pointer'lar bu sorunu çözer:

```
1  #include <stdio.h>
2
3  // Üç değeri aynı anda güncelleyen fonksiyon
4  void motorDurumOku(float *hiz, float *sicaklik, float *akim) {
5      *hiz      = 1450.0;    // RPM
6      *sicaklik = 72.5;      // °C
7      *akim     = 3.2;       // Amper
8  }
9  int main() {
10     float h, s, a;
11     motorDurumOku(&h, &s, &a); // Üç adres gönder
12     printf("Hiz: %.0f RPM\n", h);
13     printf("Sicaklik: %.1f C\n", s);
14     printf("Akim: %.1f A\n", a);
15     return 0;
16 }
```

Mekatronik Uygulaması: Gömülü sistemlerde tek bir I²C/SPI okuma fonksiyonuyla birden fazla sensör verisini eş zamanlı döndürmek bu teknikle mümkün olur.

["İleri Seviye Fonksiyonlar"]

1.5 RAM Tasarrufu: Büyük Veri Yapılarını Kopyalamadan İletmek

Büyük sensör dizilerini fonksiyona kopyalamak yerine adresini göndermek ciddi RAM tasarrufu sağlar.

```
1  #include <stdio.h>
2  #define SENSOR_SAYISI 1000
3
4  // Diziyi kopyalamadan, sadece adresini alır (RAM tasarrufu)
5  float ortalamaHesapla(float *dizi, int boyut) {
6      float toplam = 0;
7      for (int i = 0; i < boyut; i++) {
8          toplam += dizi[i]; // *(dizi + i) ile eşdeğer
9      }
10     return toplam / boyut;
11 }
12
13 int main() {
14     float sensorVerileri[SENSOR_SAYISI];
15     // ... sensörlerden veri doldurulduğunu varsayalım ...
16
17     // Dizi zaten adres olarak geçer – 1000 float kopyalanmaz!
18     float ort = ortalamaHesapla(sensorVerileri, SENSOR_SAYISI);
19     printf("Ortalama: %.2f\n", ort);
20     return 0;
21 }
```

["İleri Seviye Fonksiyonlar"]

Yöntem	RAM Kullanımı (1000 float)	Hız
Kopya ile geçirme	~4000 byte ekstra	Yavaş
Pointer ile geçirme	~8 byte (sadece adres)	Hızlı

BÖLÜM 2

Bellek Sınıfları

`static` ve `extern`

2.1 Fonksiyonların Hafızasızlık (Amnesia) Problemi

Standart bir fonksiyondaki yerel değişken, fonksiyon her çağrıldığında belleğin (Stack) içinde yeniden oluşturulur. Ancak fonksiyonun görevi bitip süslü parantezi (`}`) kapandığı an, yerel kapsam (Local Scope) kuralları gereği bellekten tamamen silinir ve taşıdığı değer unutulur. Bu duruma fonksiyonun hafızasızlık (*amnesia*) problemi denilebilir.

× Hatalı Bir Sayıcı: Bir tekerlek tur sensörünün (Enkoder) kaç kez çağrıldığını sayan fonksiyon:

```
1 void turRaporla() {
2     int sayac = 0; // Her çağrıda YENİDEN SIFIRA döner!
3     sayac++;
4     printf("Tekerlek %d tur attı.\n", sayac);
5 }
6
7 int main() {
8     turRaporla(); // Çıktı: 1
9     turRaporla(); // Çıktı: 1
10    turRaporla(); // Çıktı: 1 - Hep 1!
11    return 0;
12 }
```

Global değişken yaparsak tüm kodların erişimine açılır → güvenlik riski! Çözüm?

["İleri Seviye Fonksiyonlar"]

2.2 Çözüm: static Local Variable

`static` anahtar kelimesi değişkeni RAM'de kalıcı yapar ama dışarıdan erişilemez tutar.

✓ Zeki Sensör (Kendini Hatırlayan):

```
1 void turRaporla() {
2     static int sayac = 0; // İlk çağrıda 0 atanır, sonraki çağrılarda değer korunur
3     sayac++;
4     printf("Tekerlek %d tur attı.\n", sayac);
5 }
6
7 int main() {
8     turRaporla(); // Çıktı: 1
9     turRaporla(); // Çıktı: 2
10    turRaporla(); // Çıktı: 3 ✓ Hatırlıyor!
11    return 0;
12 }
```

Karşılaştırma Tablosu

Özellik	Normal Yerel	<code>static</code> Yerel	Global
Ömür	Blok süresi	Program süresi	Program süresi
Görünürlük	Blok içi	Blok içi	Tüm dosya
Güvenlik	✓	✓	×

Mekatronik Uygulaması: PID kontrol algoritmasında "Önceki Hata" ve "İntegral Toplamı" değerlerini saklamak için `static` kullanılır.

2.3 static ile PID Kontrol Hafızası

PID (Proportional - Integral - Derivative) Kontrol Nedir? PID, mekatronik sistemlerde (drone, robot kol, motor hız kontrolü vb.) bir donanımı istenen hedef değere (setpoint) pürüzsüzce ulaştırmak için kullanılan geri beslemeli bir matematiksel algoritmadır.

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

Üç temel bileşenden oluşur:

- P (Oransal): Sistemin mevcut anlık hatasına tepki verir.
- I (İntegral): Geçmişteki hataların toplamını tutarak sistemdeki kalıcı sapmaları sıfırlar. *(Birikimli toplamı hatırlamak için static değişkene ihtiyaç duyar).*
- D (Türevsel): Hatanın değişim hızına bakarak gelecekteki salınımı önler ve frenleme yapar. *(Bir önceki hatayı hatırlamak için static değişkene ihtiyaç duyar).*

Örnek: PID Kontrol Fonksiyonu

```
1 float pidHesapla(float hedef, float olcum) {
2     static float oncekiHata = 0.0;
3     static float integralToplam = 0.0;
4
5     float Kp = 1.2, Ki = 0.05, Kd = 0.3;
6     float hata = hedef - olcum;
7
8     integralToplam += hata;           // Birikimli toplam (hatırlıyor!)
9     float turev = hata - oncekiHata; // Önceki hatayı biliyor!
10    oncekiHata = hata;                // Güncelle ve sakla
11
12    return (Kp * hata) + (Ki * integralToplam) + (Kd * turev);
13 }
```

⚠ Global değişken kullansaydık: Başka bir fonksiyon `oncekiHata` değerini yanlışlıkla ezebilir ve kontrol sistemi kararsız hale gelebilirdi!

2.4 extern ile Çoklu Dosya Veri Paylaşımı

Büyük projelerde kod birden fazla `.c` dosyasına bölünür. `extern` anahtar kelimesi, bir değişkenin veya fonksiyonun başka bir dosyada tanımlı olduğunu bildirir.

```
1 // ===== config.c =====
2 int sistemFrekansi = 16000000; // 16 MHz (tanımlama burada)
3
4 // ===== motor.c =====
5 extern int sistemFrekansi; // "Bu değişken başka dosyada var" bildirimi
6
7 void motorBaslat() {
8     int pwmPeriyot = sistemFrekansi / 1000; // Frekansa göre PWM hesabı
9     printf("PWM Periyot: %d\n", pwmPeriyot);
10 }
11
12 // ===== main.c =====
13 extern int sistemFrekansi;
14 extern void motorBaslat(); // Fonksiyon da extern ile bildirilebilir
15
16 int main() {
17     printf("Sistem: %d Hz\n", sistemFrekansi);
18     motorBaslat();
19     return 0;
20 }
```

["İleri Seviye" Fonksiyonlar]

<https://github.com/hyuce>

17 / 55



BÖLÜM 3

Özyinelemeli (Recursive) Fonksiyonlar

3.1 Fonksiyonun Kendini Çağırması

Recursive (Özyineli) Fonksiyonlar, tekrar eden bazı problemlerde fonksiyonun kendi kendini çağırması temeline dayanır. Matematiksel karşılığı Faktöriyel ile meşhurdur: $5! = 5 \times 4!$

```
1  int faktoriyel(int n) {
2      if (n <= 1)
3          return 1;          // Durma (Base Case) Koşulu – ÇOK ÖNEMLİ!
4      else
5          return n * faktoriyel(n - 1);  // Kendini çağırır
6  }
7
8  int main() {
9      printf("5! = %d\n", faktoriyel(5));  // 120
10     return 0;
11 }
```

```
1  Çağrı Zinciri:
2  faktoriyel(5) → 5 * faktoriyel(4)
3                → 4 * faktoriyel(3)
4                → 3 * faktoriyel(2)
5                → 2 * faktoriyel(1)
6                → return 1 ← Durma noktası
```

3.2 Stack Overflow Tehlikesi — Mikrodenetleyicilerde Risk

Recursive fonksiyonda her çağrı Stack'te yeni bir frame oluşturur. Fonksiyon işini bitirmeden klonunu çağırdığı için Stack sürekli büyür.

× Donanım Tehlikesi:

Platform	Stack Boyutu	Recursive Çağrı Limiti
PC (x86_64)	~8 MB	~100.000+
STM32F4	~8 KB	~200-500
ATmega328P (Arduino)	2 KB	~50-100

1 Stack Belleğinin Dolması:

2

3 faktoriyel(1) ← 5. çağrı

4 faktoriyel(2) ← 4. çağrı

5 faktoriyel(3) ← 3. çağrı

6 faktoriyel(4) ← 2. çağrı

7 faktoriyel(5) ← 1. çağrı

8 main()

⚠ Stack doldu!
→ Hard Reset / Çökme!

["İleri Seviye Fonksiyonlar"]

3.3 Çözüm: İteratif (Döngüsel) Dönüşüm

Mekatronikte recursive fonksiyonlardan kaçınılmalı; her zaman döngüsel (iterative) alternatif tercih edilmelidir.

```
1 // ❌ Recursive – Stack riski taşır
2 int faktoriyel_r(int n) {
3     if (n <= 1) return 1;
4     return n * faktoriyel_r(n - 1);
5 }
6
7 // ✅ İteratif – Sabit Stack kullanımı, güvenli
8 int faktoriyel_i(int n) {
9     int sonuc = 1;
10    for (int i = 2; i <= n; i++) {
11        sonuc *= i;    // Aynı RAM adresi üzerinde çalışır
12    }
13    return sonuc;
14 }
```

3.4 Karşılaştırma Tablosu

Kriter	Recursive	İteratif
Okunabilirlik	Matematiksel formüle yakın	Biraz daha uzun
Stack Kullanımı	$O(n)$ — tehlikeli	$O(1)$ — sabit
Hız	Çağrı yükü var	Daha hızlı
Mekatronik Uygunluk	× Riskli	✓ Güvenli

Mekatronik Prensibi: Gömülü donanımda recursive fonksiyonlardan uzak dur; `for` / `while` döngülerini tercih et.

BÖLÜM 4

Yüksek Performanslı Çağrılar:

`inline` Fonksiyonlar

4.1 Fonksiyon Çağrı Yükü (Call Stack Overhead)

Her fonksiyon çağrısında işlemci şu adımları uygular:

1. Parametreleri Stack'e yaz (push)
2. Dönüş adresini kaydet
3. Program sayacını fonksiyona yönlendir (jump)
4. İş tamamlanınca geri yükle (pop) ve dön

Bu süreç normalde önemsizdir. Ancak 1 saniyede milyonlarca kez çağrılan küçük fonksiyonlarda (motor kontrol döngüsü) bu zıplamalar sistemi yavaşlatır.

```
1 Normal Fonksiyon Çağrısı:  
2 main() —JUMP→ pinDurumu() —JUMP→ main()  
3     ~10-20 CPU cycle kaybı (her çağrıda)  
4  
5 Motor kontrol döngüsü: 10 kHz × 10 fonksiyon = 100.000 zıplama/saniye!
```

4.2 inline Fonksiyon Mimarisi (C99)

`inline` anahtar kelimesi derleyiciye "Bu fonksiyonu çağırma, gövdesini doğrudan çağrı noktasına yapıştır" ricasında bulunur.

```
1 // Derleyici bu fonksiyonun gövdesini çağrı noktasına enjekte edecektir
2 inline int pinDurumuTersCevir(int x) {
3     return x == 1 ? 0 : 1;
4 }
5
6 int main() {
7     int led = 1;
8     // Derleyici bunu arkaplanda şuna çevirir:
9     // led = (led == 1 ? 0 : 1);    ← Zıplama yok!
10    led = pinDurumuTersCevir(led);
11    return 0;
12 }
```

4.3 `inline` Fonksiyon vs. Normal Fonksiyon

Kriter	Normal Fonksiyon	<code>inline</code> Fonksiyon
Çağrı yükü	Var (~10-20 cycle)	Yok (0 cycle)
Kod boyutu	Küçük (tek kopya)	Büyür (her çağrıda kopya)
Tip güvenliği	✓ Var	✓ Var
Uygun kullanım	Büyük fonksiyonlar	Küçük, sık çağrılan fonksiyonlar

⚠ Not: `inline` bir garanti değil, derleyiciye bir ricadır. Derleyici uygun görmezse normal çağrı yapar.

BÖLÜM 5

Ön İşlemci Seviyesinde Optimizasyon: Makro Fonksiyonlar (`#define`)

5.1 Parametrel Makro Oluşturma

C dilinde derleme işlemi başlamadan önce "Ön İşlemci" (Preprocessor) adı verilen bir mekanizma çalışır. `#define` ile tanımlanan makrolar gerçek birer fonksiyon değildir. Ön işlemci, kodda makroyu nerede görürse, tıpkı bir metin editöründe "Bul ve Değiştir" yapmışsınız gibi, makronun içindeki ifadeyi oraya düz metin olarak yerleştirir. Ortada gerçek bir fonksiyon çağırısı olmadığı için CPU zıplaması yaşanmaz ve işlem sıfır gecikme (zero-overhead) ile çalışır.

```
1 // Parametrel makro: Derlenmeden önce metin olarak yerine konur
2 #define KUP_AL(x) ((x) * (x) * (x))
3 #define MAX(a, b) ((a) > (b) ? (a) : (b))
4
5 int main() {
6     float sonuc = KUP_AL(5); // Derleyici: ((5) * (5) * (5)) = 125
7     int buyuk = MAX(10, 20); // Derleyici: ((10) > (20) ? (10) : (20)) = 20
8     return 0;
9 }
```

5.2 Makro Riskleri

Özellikle matematiksel hesaplamalar içeren makrolarda dikkat edilmesi gereken hayati kurallar vardır. C dilinde derleyici, makro parametrelerini alıp sadece metin olarak yerine koyduğu için işlem sırası (operator precedence) kontrolü yapmaz. Bu durum, parantez kullanılmadığında ciddi matematiksel hatalara yol açar.

⚠ Parantez Kuralı: Her parametre ve tüm ifade mutlaka parantez içine alınmalıdır! Aksi halde matematiksel hataları oluşur:

```
1 // ❌ Parantez eksik – TEHLİKELİ
2 #define KARE_HATALI(x) x * x
3 int sonuc = KARE_HATALI(3 + 2); // 3 + 2 * 3 + 2 = 11 (beklenen: 25!)
4
5 // ✅ Doğru tanım
6 #define KARE(x) ((x) * (x))
7 int sonuc2 = KARE(3 + 2); // ((3 + 2) * (3 + 2)) = 25 ✅
```

5.3 Makro vs. `inline` Karşılaştırması

Kriter	<code>#define</code> Makro	<code>inline</code> Fonksiyon
İşlem zamanı	Derleme öncesi (preprocessor)	Derleme sırasında
Tip kontrolü	× Yok — tür güvenliği riski	✓ Var — derleyici kontrol eder
Hata ayıklama	Zor — makro genişletilmiş kodu gösterir	Kolay — normal fonksiyon gibi
Çağrı yükü	Yok	Yok
Uygun kullanım	Bit işlemleri, sabitler	Küçük hesaplama fonksiyonları

5.4 Donanım Yazmaçlarında Makro ile Bit İşlemleri

Mikrodenetleyici register'larında mikrosaniye hızında bit set/reset işlemleri için makrolar vazgeçilmezdir:

```
1 // STM32 benzeri register bit işlem makroları
2 #define BIT_SET(REG, BIT) ((REG) |= (1U << (BIT))) // Biti 1 yap
3 #define BIT_CLEAR(REG, BIT) ((REG) &= ~(1U << (BIT))) // Biti 0 yap
4 #define BIT_TOGGLE(REG, BIT) ((REG) ^= (1U << (BIT))) // Biti tersle
5 #define BIT_READ(REG, BIT) (((REG) >> (BIT)) & 1U) // Biti oku
6
7 // Kullanım örneği (GPIO port kontrol):
8 unsigned int PORTB = 0x00;
9
10 BIT_SET(PORTB, 5); // Pin 5'i HIGH yap (LED yak)
11 BIT_CLEAR(PORTB, 5); // Pin 5'i LOW yap (LED söndür)
12 BIT_TOGGLE(PORTB, 3); // Pin 3'ü tersle
13
14 if (BIT_READ(PORTB, 7)) { // Pin 7 HIGH mı?
15     // Buton basılı...
16 }
```

Mekatronik Uygulaması: Bu makrolar, motor sürücü IC'lerinin kontrol yazmaçlarına doğrudan ve hızlı erişim sağlar. Fonksiyon çağırısı yapılmadığı için gecikme sıfırdır.

BÖLÜM 6

Esnek Veri Yapıları: Değişken Sayıda Parametre (Variadic Functions)

6.1 `stdarg.h` Kütüphanesi ve Çalışma Mantığı

Şimdiye kadar yazdığımız fonksiyonların parametre sayısı her zaman sabitti (Örn: `topla(int a, int b)` daima 2 değer bekler). Ancak C dilinde bazı özel fonksiyonlar, her çağrıldığında birbirinden farklı sayıda argüman kabul edebilir. Bunun en meşhur örneği; bazen tek bir metin, bazen de 5 farklı değişkeni aynı anda ekrana yazdırabilen `printf` fonksiyonudur.

Kendi fonksiyonlarımıza bu esnekliği kazandırmak için `stdarg.h` kütüphanesini kullanırız. Fonksiyonun değişken sayıda parametre alabileceğini derleyiciye bildirmek için parametre listesinin sonuna üç nokta `...` (ellipsis) sembolü konur.

6.2 `stdarg.h` Makrolarının Görevleri

Makro	Ne İşe Yarar?
<code>va_list</code>	Liste Değişkeni: Gönderilen ekstra parametreleri (üç noktaya karşılık gelenleri) bellekte toplu halde tutacak olan değişken türüdür.
<code>va_start(liste, sonSabit)</code>	Listeyi Başlatma: Parametreleri okumaya başlamadan önce belleği hazırlar. <code>sonSabit</code> , üç noktadan (<code>...</code>) hemen önceki son normal parametrenin adıdır.
<code>va_arg(liste, tip)</code>	Sıradaki Değeri Okuma: Listedeki sıradaki parametreyi alır ve belirtilen veri türüne (örn: <code>int</code> , <code>float</code>) çevirir. Her çağrıldığında otomatik olarak bir sonraki parametreye geçer.
<code>va_end(liste)</code>	Temizlik (Kapatma): Tüm parametreler okunduktan sonra, arka planda kullanılan bellek alanını serbest bırakarak işlemi güvenli bir şekilde sonlandırır.

6.3 Örnek: Değişken Argümanlı Fonksiyon

```
1  #include <stdio.h>
2  #include <stdarg.h>
3
4  // İlk parametre (kacTane), kendisinden sonra kaç tane sayı geleceğini söyler.
5  int sayilariTopla(int kacTane, ...) {
6      va_list liste;                // ADIM 1: Ekstra argümanlar için boş liste oluştur
7      va_start(liste, kacTane);     // ADIM 2: Listeyi 'kacTane' değişkeninden hemen sonra başlat
8
9      int toplam = 0;
10     for (int i = 0; i < kacTane; i++) {
11         // ADIM 3: Listedeki sıradaki veriyi "int" (tamsayı) formatında çek al
12         int siradakiSayi = va_arg(liste, int);
13         toplam += siradakiSayi;
14     }
15
16     va_end(liste);                 // ADIM 4: Temizlik yap (belleği iade et)
17     return toplam;
18 }
19
20 int main() {
```

6.4 Mekatronik Uygulaması: Özel UART Hata Loglama Fonksiyonu

Haberleşme protokolleri üzerinden dinamik hata mesajları göndermek için `printf` benzeri özelleştirilmiş fonksiyon:

```
1  #include <stdio.h>
2  #include <stdarg.h>
3
4  // Seviyeler -> 0: BILGI, 1: UYARI, 2: HATA
5  void hataBildir(int seviye, const char *metinSablonu, ...) {
6      // ADIM 1: Mesajın başına gelecek tehlike etiketini seç
7      if (seviye == 0)      printf("[BILGI] ");
8      else if (seviye == 1) printf("[UYARI] ");
9      else if (seviye == 2) printf("[HATA] ");
10
11     // ADIM 2: Değişken sayıda parametreler için listeyi hazırla
12     va_list liste;
13     va_start(liste, metinSablonu);
14
15     // ADIM 3: 'vprintf' özel bir hazır fonksiyondur.
16     // Hazırladığımız listeyi alır ve %d, %f gibi yer tutucuların içine otomatik yerleştirir.
17     vprintf(metinSablonu, liste);
18
19     // ADIM 4: Temizlik yap ve mesaj bitince alt satıra geç
20     va_end(liste);
21 }
```

["İleri Seviye Fonksiyonlar"]

Beklenen Çıktı:

```
1 [BILGI] Sistem basariyla baslatildi.  
2 [UYARI] Motor 2 sicakligi biraz yuksek: 95.3 C  
3 [HATA] Motor 2 ASIRI ISINMA! Acil stop...
```

BÖLÜM 7

Dinamik Mimari: Fonksiyon İşaretçileri (Function Pointers)

7.1 Fonksiyon İşaretçisi Nedir?

Şimdiye kadar pointer'ları hep verilerin (değişkenlerin) RAM'deki yerini göstermek için kullandık. Ancak program hafızasındaki (Code/Text Segment) fonksiyonların da kendilerine ait fiziksel birer bellek adresi vardır.

Nasıl ki bir haritada binaların koordinatları varsa, derleyici de yazdığımız her fonksiyonu bellekte belirli bir başlangıç adresine yerleştirir. Fonksiyon İşaretçisi (Function Pointer), verilerin değil, bizzat *algoritmanın* (fonksiyonun) bellekteki bu başlangıç adresini tutan özel bir yönlendiricidir.

Bu yapı sayesinde yazılım çalışırken "Acaba şimdi hangi fonksiyonu çalıştırsam?" kararını karmaşık if-else yığınlarına girmeden dinamik olarak (çalışma zamanında) verebiliriz.

7.1.1 Kavramsal Sözdizimi (Syntax) Karşılaştırması

- Normal Pointer: `int *ptr;` *(Sıradan bir tamsayının bellekteki fiziksel adresini tutar.)*
- Fonksiyon Pointer: `int (*ptr)(int, int);` *(İki adet tamsayı alıp, sonuç olarak tamsayı döndüren bütün fonksiyonların başlangıç adresini tutabilir.)*

7.2 Örnek: Fonksiyon İşaretçisi Tanımı ve Kullanımı

```
1  #include <stdio.h>
2
3  int topla(int a, int b) { return a + b; }
4  int cikar(int a, int b) { return a - b; }
5
6  int main() {
7      // Fonksiyon işaretçisi tanımı:
8      // int (*fp)(int, int) → "iki int alıp int döndüren fonksiyona işaretçi"
9      int (*fp)(int, int);
10
11     fp = topla;                // topla fonksiyonunun adresini ata
12     printf("Toplam: %d\n", fp(5, 3)); // 8
13
14     fp = cikar;                // Aynı işaretçi, farklı fonksiyon!
15     printf("Fark: %d\n", fp(5, 3)); // 2
16     return 0;
17 }
```

Sözdizimi: `DönüşTipi (*işaretçiAdı)(ParametreTipleri)`

7.2 State Machine (Durum Makinesi) Tasarımı

Otonom sistemlerde `switch-case` yapılarını yerine fonksiyon işaretçileri ile hızlı ve temiz durum makineleri tasarlanabilir:

```
1  #include <stdio.h>
2
3  // Durum fonksiyonları
4  void durumBekle() { printf(">> Bekleme modunda...\n"); }
5  void durumTara() { printf(">> Ortam taranıyor...\n"); }
6  void durumHareket() { printf(">> Hedefe hareket ediliyor...\n"); }
7  void durumDur() { printf(">> Acil durak!\n"); }
8
9  int main() {
10     // Fonksiyon işaretçi dizisi - durumları indeksleme
11     void (*durumlar[])() = { durumBekle, durumTara, durumHareket, durumDur };
12
13     int senaryo[] = {0, 1, 2, 1, 3}; // Durum geçiş sırası
14     int adimSayisi = 5;
15
16     for (int i = 0; i < adimSayisi; i++) {
17         durumlar[senaryo[i]](); // İndeks ile doğrudan fonksiyon çağırısı
18     }
19     return 0;
20 }
```

["İleri Seviye Fonksiyonlar"]

BÖLÜM 8

`main` Fonksiyonuna Parametre Geçme

8.1 main Fonksiyonunun Tam Anatomisi

`main()` fonksiyonu, işletim sisteminden komut satırı argümanları alabilir:

```
1 int main(int argc, char *argv[]) {  
2     // argc: Argüman sayısı (argument count)  
3     // argv: Argüman dizisi (argument vector)  
4     // argv[0] = Program adı  
5     // argv[1] = İlk argüman  
6     // argv[2] = İkinci argüman  
7     // ...  
8     return 0;  
9 }
```

1 Terminal komutu: `./robot_kontrol COM3 kalibrasyon.dat`

2		
3		
4	argc	3
5		
6	argv[0]	"./robot_kontrol"
7	argv[1]	"COM3"
8	argv[2]	"kalibrasyon.dat"
9		

8.2 Argüman Ayırıştırma (Parsing) ve `atoi`

Terminalden gelen tüm parametreler Metin (String) formatındadır. Eğer bunları sayısal işlemlerde kullanacaksak, tamsayıya (Integer) çeviren `atoi()` (ASCII to Integer) fonksiyonuna ihtiyacımız vardır.

```
1  #include <stdio.h>
2  #include <stdlib.h> // atoi() fonksiyonu için gereklidir
3
4  // Terminalden Motor Numarasını ve Hızını alarak sistemi başlatan örnek
5  int main(int argc, char *argv[]) {
6      // ADIM 1: Kullanıcı yeterli parametre girmiş mi kontrolü
7      // (1 Programın adı + 2 Parametre = Toplam 3 argüman olmalı)
8      if (argc != 3) {
9          printf("HATA: Eksik parametre girdiniz!\n");
10         printf("Dogru kullanım: %s <Motor_No> <Hedef_Hiz>\n", argv[0]);
11         printf("Ornek komut   : %s 5 1500\n", argv[0]);
12         return 1; // Sistemi hata koduyla durdur
13     }
14
15     // ADIM 2: Terminalden gelen char* (metin) dizilerini, 'atoi' ile tam sayıya çevir
16     int motorNo = atoi(argv[1]);
17     int hedefHiz = atoi(argv[2]);
18
19     // ADIM 3: Ayırıştırılan (Parse edilen) verileri donanıma gönder
20     printf("=== Donanım Bağlantısı Kuruldu! ===\n");
```

8.2.1 Derleme ve Çalıştırma:

```
1 $ gcc motor_kontrol.c -o motor_kontrol
2 $ ./motor_kontrol 5 1500
3 >>> Donanım Bağlantısı Kuruldu!
4 >>> Motor 5 aktif ediliyor...
5 >>> Hedef hız 1500 RPM olarak ayarlandı.
```

8.3 Mekatronik Uygulaması: Başlangıç Parametreleri

Gömülü Linux sistemlerinde (Raspberry Pi, Jetson Nano) çalışan kontrol yazılımlarına başlatma anında donanım bilgisi aktarılabilir:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main(int argc, char *argv[]) {
6      int hiz = 50;           // Varsayılan motor hızı (%)
7      char *port = "/dev/ttyUSB0"; // Varsayılan port
8
9      // Argümanları kontrol et
10     for (int i = 1; i < argc; i++) {
11         if (strcmp(argv[i], "--hiz") == 0 && i + 1 < argc) {
12             hiz = atoi(argv[i + 1]); // String → int dönüşümü
13             i++; // Bir sonraki argümanı atla
14         }
15         else if (strcmp(argv[i], "--port") == 0 && i + 1 < argc) {
16             port = argv[i + 1];
17             i++;
18         }
19     }
20 }
```

["İleri Seviye Fonksiyonlar"]

Özet

Bu Hafta Öğrendiklerimiz (İleri Seviye Fonksiyonlar):

Konu	Temel Kazanım
Pointer / Pass by Reference	Bellek adresi ile çağırma, birden fazla değer döndürme, RAM tasarrufu
<code>static</code> / <code>extern</code>	Durum (state) koruma, çoklu dosya modülerliği
Recursive Fonksiyonlar	Kendi kendini çağırma, Stack Overflow riski, iteratif dönüşüm
<code>inline</code> Fonksiyonlar	Çağrı yükünü ortadan kaldırma, hız-boyut dengesi
Makro Fonksiyonlar	Preprocessor düzeyinde optimizasyon, register bit işlemleri
Variadic Functions	<code>stdarg.h</code> ile değişken sayıda parametre, özel loglama
Fonksiyon İşaretçileri	State Machine, Callback mimarisi
<code>main</code> Parametreleri	<code>argc</code> / <code>argv</code> ile komut satırı argüman yönetimi

Artık mekatronik sistemlerin yazılım mimarisini kuracak ileri seviye C fonksiyon araçlarına sahipsiniz!

Laboratuvar Örnekleri (Bölüm 1)

Uygulama 1: Çoklu Sensör Okuma (Pointer)

Amaç: `Pass by Reference` kullanarak tek fonksiyondan birden fazla değer döndürmek. **Görev:** `sensorleriOku(&isi, &nem, &basinc)` adında bir fonksiyon yazın. Bu fonksiyon, bellek adreslerini aldığı bu üç değişkene sırasıyla `25.4`, `60.2` ve `1013` değerlerini atasın. `main` içinde değerleri ekrana yazdırın.

Uygulama 2: PID Hafıza Simülasyonu (`static`)

Amaç: `static` yerel değişkenlerin "hafıza" (amnesia çözümü) özelliğini kavramak. **Görev:** `hizFarkiHesapla(int guncelHiz)` adında bir fonksiyon yazın. Fonksiyon, bir önceki çağrılışındaki hız değerini `static` bir değişkende saklasın ve yeni gelen hız ile eski hız arasındaki farkı (ivmeyi) ekrana yazdırsın. `main` içinde ardışık olarak 3 farklı hız gönderip test edin.

Laboratuvar Örnekleri (Bölüm 2)

Uygulama 3: Akıllı Ev Komut Sistemi (`argc` , `argv` , `atoi`)

Amaç: Terminal argümanlarını ayrıştırmak ve sayıya çevirmek. Görev: Programınız terminalden iki parametre alsın: `./akilli_ev 3 24` . `main` içinde bu metinleri `atoi` ile tam sayıya çevirin ve *"3 numaralı oda 24 dereceye ayarlanıyor..."* şeklinde çıktı verin. Eğer eksik parametre girilirse program kullanıcıya "Hatalı kullanım" uyarısı versin.

Uygulama 4: Acil Durum Yönetimi (Function Pointers)

Amaç: Fonksiyon işaretçileri ile dinamik (çalışma zamanı) karar mekanizması kurmak. Görev: `motoruDurdur()` ve `alarmiCal()` adlı parametre almayan iki basit fonksiyon yazın. Sonrasında parametre olarak bir fonksiyon işaretçisi alan `acilDurumTetikle( ... )` isimli ayrı bir fonksiyon oluşturun. `main` içinden hangi fonksiyonun adresini gönderirseniz o görevin çalışmasını sağlayın.

Laboratuvar Çözümleri (1 ve 2)

```
1 // Uygulama 1 Çözümü: Çoklu Sensör Okuma
2 #include <stdio.h>
3 void sensorleriOku(float *isi, float *nem, int *basinc) {
4     *isi = 25.4;
5     *nem = 60.2;
6     *basinc = 1013;
7 }
8 int main() {
9     float s1, s2; int s3;
10    sensorleriOku(&s1, &s2, &s3);
11    printf("Isi: %.1f, Nem: %.1f, Basinc: %d\n", s1, s2, s3);
12    return 0;
13 }
```

```
1 // Uygulama 2 Çözümü: PID Hafıza Simülasyonu
2 #include <stdio.h>
3 void hizFarkiHesapla(int guncelHiz) {
4     static int oncekiHiz = 0; // İlk çağrılıştta 0 olur, sonra değerini korur
5     int ivme = guncelHiz - oncekiHiz;
6     oncekiHiz = guncelHiz;
7     printf("Guncel: %d, Degisim: %d\n", guncelHiz, ivme);
8 }
9 int main() {
10     hizFarkiHesapla(10); hizFarkiHesapla(25); hizFarkiHesapla(25);
11     return 0;
12 }
```

Laboratuvar Çözümleri (3 ve 4)

```
1 // Uygulama 3 Çözümü: Akıllı Ev Komut Sistemi
2 #include <stdio.h>
3 #include <stdlib.h>
4 int main(int argc, char *argv[]) {
5     if(argc != 3) {
6         printf("Hatali kullanim!\n");
7         return 1;
8     }
9     int oda = atoi(argv[1]);
10    int sicaklik = atoi(argv[2]);
11    printf("%d numarali oda %d dereceye ayarlaniyor...\n", oda, sicaklik);
12    return 0;
13 }
```

```
1 // Uygulama 4 Çözümü: Acil Durum Yönetimi
2 #include <stdio.h>
3 void motoruDurdur() { printf("Motor ACIL durduruldu!\n"); }
4 void alarmiCal() { printf("DIIIT! Alarm aktif!\n"); }
5
6 void acilDurumTetikle(void (*islem)(void)) {
7     printf("Acil Protokol: ");
8     islem(); // Gelen fonksiyonu çalıştır
9 }
10 int main() {
11     acilDurumTetikle(motoruDurdur);
12     acilDurumTetikle(alarmiCal);
13     return 0;
14 }
```

Teşekkürler...

Dr. Öğr. Üyesi Hüseyin YÜCE

Marmara Üniversitesi

Mekatronik Mühendisliği Bölümü